WEIR - Assignment 2 - Document Retrieval

Maj Bregar, Gal Fajon, Alen Leban
Faculty of Computer and Information Science
Večna pot 113, 1000 Ljubljana

***Abstract*—This assignment builds on the PA1 crawl by extracting meaningful content from HTML pages, segmenting it into coherent text segments, encoding them with a sentence embedding model, and storing them in a pgvector-enabled PostgreSQL database for semantic retrieval.**

We implement parsing tailored to 24ur.com articles, apply sentence-aware chunking, index embeddings with HNSW, and provide a query and reranking pipeline based on vector similarity and a cross-encoder reranker.

I. Introduction

This report describes the design and implementation of a parsing and semantic embedding pipeline that processes the HTML pages collected by the preferential web crawler developed in the previous assignment. The goal is to extract clean article text from the crawled pages and represent it in a form that supports efficient domain-specific information retrieval.

The pipeline consists of two main stages: preprocessing and retrieval. During preprocessing, crawled HTML pages from 24ur.com are parsed, and the extracted text is divided into sentence-based segments. The resulting segments are then embedded using a transformer-based sentence embedding model and stored in a vector database.

During retrieval, a user query is embedded using the same embedding model and used to retrieve the most similar segments from the database. Query–segment pairs are subsequently evaluated by a reranking model, which refines the initial ranking based on semantic relevance. The final output of the pipeline is a ranked list of text segments that are most semantically relevant to the query.

II. Methodology

A. Website filtering and data source

We reuse the PA1 crawl database from the previous assignment and restrict processing to HTML pages with non-empty `html_content`. In our pipeline this filtering is performed in a database query that fetches page IDs and HTML bodies.

B. HTML parsing with XPath and regural expressions

HTML parsing is implemented with `lxml`. We extract the article title by first reading the OpenGraph metadata (`//meta[@property='og:title']/@content`); if missing, we fall back to `//title`. A regex post-process removes the site suffix "`| 24ur.com`" and normalizes whitespace. The main article body is extracted from the `#article-body` node using XPath, and non-content elements (`script`, `style`, `noscript`, `img`, `figure`, `svg`, `picture`) are removed to keep only the textual content.

We additionally capture the lead summary by locating the main article picture and the adjacent summary paragraph via XPath, then prepend it to the article body. A regex is used for lightweight text normalization (whitespace collapsing) and title cleanup.

This hybrid approach keeps extraction robust while remaining lightweight and transparent.

C. Chunking

After parsing the crawled web pages, the extracted text is divided into smaller sentence-based segments. Sentence boundaries are detected using the NLTK sentence tokenizer configured for the Slovene language. Segments are constructed incrementally by appending consecutive sentences until a predefined maximum word limit is reached. Once the limit is exceeded, a new segment is created. This approach preserves local semantic context while ensuring that segment sizes remain suitable for transformer-based embedding models.

D. Segment Embedding

Document segments were transformed into dense vector representations using transformer-based sentence embedding models. Each segment was embedded independently, and the resulting vectors were stored in dedicated database tables grouped by embedding dimensionality. We extend the PA1 database with a `cleaned_content` column and three segment tables for different embedding sizes: `page_segment_vec384`, `page_segment_vec768`, and `page_segment_vec1024`. Each segment row stores the page ID, the model ID, the text segment, and the vector embedding. A separate `model` table records the embedding model name, enabling multiple models to coexist.

To ensure consistent similarity computation during retrieval, embeddings generated by all evaluated models were normalized to unit length prior to storage, despite the embedding models already producing nearly unit-length vectors.

The embedding computation included both multilingual and Slovenian-specialized embedding models. Table I summarizes the embedding models used in the experiments together with their output vector dimensionalities.

TABLE I: Embedding models used for vectorization

Model	Vec. Length
`sentence-transformers/all-MiniLM-L6-v2`	384
`sentence-transformers/all-mpnet-base-v2`	768
`EMBEDDIA/sloberta`	768
`cjvt/crosloengual-bert-si-nli`	768
`BAAI/bge-m3`	1024

To support efficient similarity-based retrieval, vector indexes were created for each embedding table using the `pgvector` PostgreSQL extension. Separate HNSW indexes were generated for cosine similarity and Euclidean (L2) distance metrics, enabling experimentation with different retrieval strategies. Additionally, standard indexes were created for embedding model identifiers to enable efficient filtering by model. Since `pgvector` does not provide indexing support for L1 distance, L1 retrieval is performed without vector indexing.

E. Segment Retrieval and Reranking

During retrieval, the user query is embedded using the same embedding model as the stored segment embeddings. The generated query vector is then used to query the vector database and retrieve the top 100 most similar segments according to the selected distance metric.

The retrieved segments are subsequently reranked using a cross-encoder model. For reranking, query–segment text pairs are constructed and processed jointly by the reranker model to produce relevance scores. The final retrieval output consists of the 10 highest-ranked segments according to the reranker scores.

Table II summarizes the reranking models used in the experiments.

TABLE II: Reranking model names

Reranker Model
`cross-encoder/ms-marco-MiniLM-L6-v2`
`BAAI/bge-reranker-v2-m3`

III. Embedding Analysis

First, we evaluated the quality of the embedding models by retrieving N closest segment embeddings to each of the M query embeddings according to the used distance metric. This provides a baseline performance measure for selecting the best-performing embedding model without introducing bias from reranking models.

To evaluate the retrieval quality of our embedding models, we employ the Normalized Discounted Cumulative Gain metric at k retrieved segments (NDCG@k). This metric is well suited for retrieval pipelines, as it measures not only the number of relevant segments among the top k retrieved results, but also whether the most relevant segments appear near the top of the ranking. For evaluation, we compute the mean NDCG@k across multiple queries, providing a more representative measure of average retrieval performance.

Queries used when computing mean NDCG@k scores were obtained by randomly picking some number of pages from the database and taking the first segment of each page. Only the first 128 characters of the chosen segments were used to extract a portion of the title. After we retrieved N closest segments to each query (for each model), ground truth relevancy score was calculated with different strategies: BM25 score, higher score for same page segments, and comparing to a reranked order using one of the reranking models.

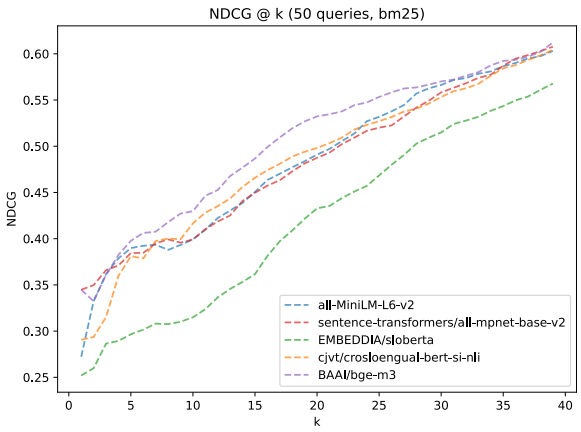

Fig. 1: Comparison of embedding models using the mean NDCG@k metric with the BM25 strategy.

For the BM25 strategy, we used ”Lucene” version of the BM25 algorithm and computed scores for each result. The top 10 highest scoring results kept their scores, while others were set to 0 to emphasise the difference between relevant and irrelevant results. Measurements using this strategy are shown in Figure 1.

The second strategy focused on giving higher scores to segments that were taken from the same page as the query. If so, that segment is given a score of 1, otherwise a score of 0.

The final strategy simply compares the ordering of results from the embedding model with the reranked ordering using the `ms-marco-MiniLM-L6-v2` model. An embedding model that produced an ordering closer to its reranked ordering is considered as better.

The embedding model performance is shown in Figure 2. The results indicate that the model `some-model` consistently outperforms the other evaluated models. Cosine similarity was used for all similarity-based retrieval experiments.

Similarly, we evaluated the performance of the reranking models by first retrieving the N most similar embeddings and then selecting the top k segments using the reranker model. For this evaluation, the performance of the embedding model `some-model` was used as the quality baseline. As shown in Figure 3, reranking significantly improves retrieval quality, with the model `some-retriever` achieving the best performance.

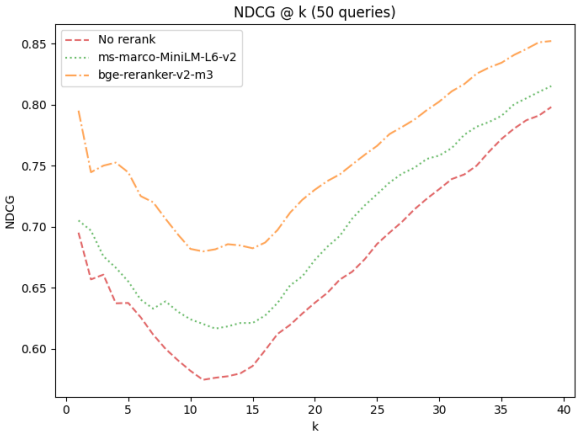

Fig. 2: Comparison of embedding models using the mean NDCG@k metric over 50 queries.

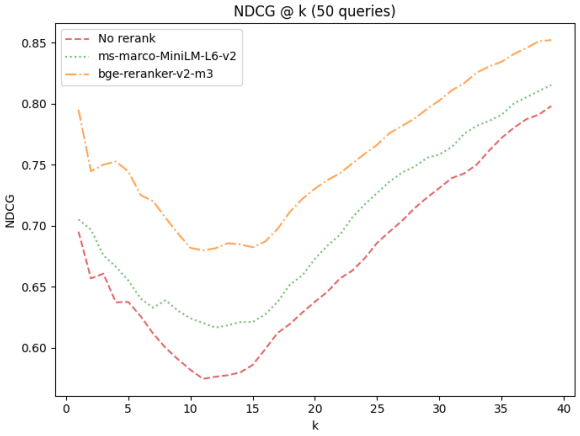

Fig. 3: Comparison of reranking models using the mean NDCG@k metric over 50 queries.

Finally, we measured the impact of the used distance metric when calculating embedding similarity. We compared cosine, L1 and L2 distance metrics on the embeddings of the best performing embedding model.

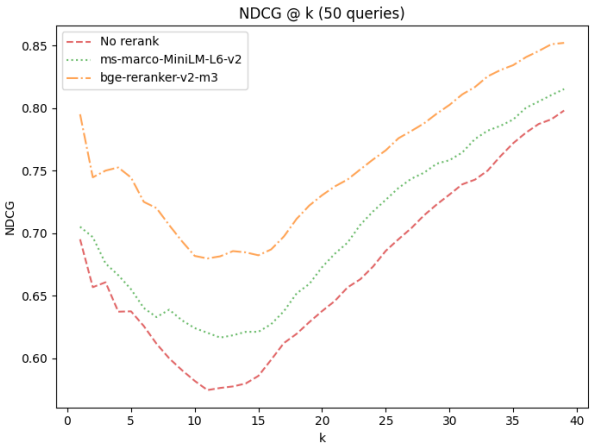

Fig. 4: Comparison of distance metric retrieval quality scored by the NDCG metric averaged over 50 different queries.

To evaluate how effectively the models group embedded segments, we visualized embeddings using UMAP. The embeddings were first reduced with SVD, retaining 40% of the variance, and then projected into a two-dimensional space using UMAP. Each point in the scatter plot represents a single segment embedding.

IV. Results

The final selected parameters of our pipeline are noted in Table III. [1]

TABLE III: Final pipeline parameters

Parameter	Chosen Value
`Embedding Model`	BAAI/bge-m3
`Reranking Model`	bge-reranker-v2-m3
`Segment Max Length`	128 words
`Retrieval Dist. Metric`	cosine
`Similarity Return n`	100
`Reranking Return k`	10

- NDCG @ k metric for final params - examples of parsed html - examples of segments

A. Retrieval Examples

- examples of successful queries - examples of unsuccessful queries

V. Disscussion

Experiments clearly show reranking significantly improves the relevancy of top results, but due to the harder interpretability of the NDCG@k metric concrete relative improvements between models are harder to determine.

- what can be done better, what we could expand on - TODO EXPAND LATER The parser is tailored to 24ur.com and relies on stable HTML structure. Pages that deviate from the `#article-body` template or heavily script-rendered content can yield empty or partial text. Chunking relies on sentence tokenization quality and may produce uneven segment sizes. Reranking improves accuracy but adds latency, which may be problematic for interactive settings.

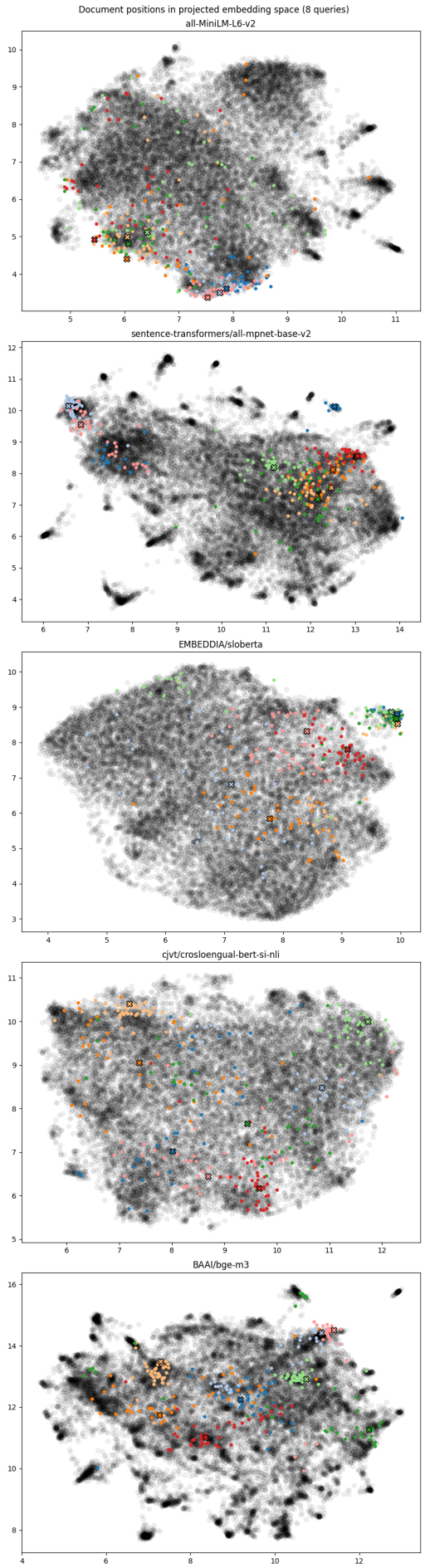

Fig. 5: UMAP visualization of segment embedding clusters.

VI. Conclusion

We implemented an end-to-end extraction and retrieval pipeline for the PA1 crawl of 24ur.com. The system parses raw HTML, chunks text using sentence boundaries, stores embeddings in a pgvector database with HNSW indexing, and retrieves and reranks results for user queries. The approach is modular, supports multiple embedding models, and can be easily configured using an `.env` file.

References

[1] J. Chen, S. Xiao, P. Zhang, K. Luo, D. Lian, and Z. Liu, "Bge m3-embedding: Multi-lingual, multi-functionality, multi-granularity text embeddings through self-knowledge distillation," *arXiv preprint arXiv:2402.03216*, vol. 4, no. 5, 2024.